

Kathmandu University

Department of Computer Science and Engineering

Dhulikhel, Kavre



A Project Report on

Problem Assessment Project (EGPG 600)

Q-Learning for Task Offloading in 3-Tier Device-Edge-Cloud Systems

A Simulation-Based Study on Mobile Edge Computing Resource Management

Submitted By

Sarayu Gautam

Masters in Computer Engineering

Submitted To

Department of Computer Science and Engineering

Kathmandu University

September 2026

DECLARATION

I hereby declare that the work reported in this project report entitled "Q-Learning for Task Offloading in 3-Tier Device-Edge-Cloud Systems", submitted to the Department of Computer Science and Engineering, Kathmandu University, in partial fulfillment of the requirements for the Problem Assessment Project (EGPG 600), is an authentic record of my own work. The simulation software, experiments and analysis presented here were carried out by me. All sources of information have been duly acknowledged, and this work has not been submitted elsewhere for the award of any other degree or diploma.

A handwritten signature in cursive script, reading "Sarayu", is written over a horizontal line.

Sarayu Gautam

Date: 2026-09-11

CERTIFICATE

This is to certify that the project report entitled "Q-Learning for Task Offloading in 3-Tier Device-Edge-Cloud Systems", submitted by Sarayu Gautam to the Department of Computer Science and Engineering, Kathmandu University, in partial fulfillment of the requirements for the Problem Assessment Project (EGPG 600), is a record of bona fide work carried out under my supervision. The report is recommended for evaluation.

Mr. Trailokya Ojha

Assistant Professor, Department of Computer Science and Engineering

Kathmandu University

Date: _____

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my supervisor for continued guidance throughout this project, and to the Department of Computer Science and Engineering, Kathmandu University, for providing the academic framework and resources that made this work possible. I am grateful to the guided course faculty for the structured progression through mobile edge computing, Markov decision processes, discrete-event simulation and reinforcement learning that underpins this report. Finally, I thank my family and peers for their support and encouragement.

Abstract

Resource-constrained devices cannot meet the computational demands of modern mobile applications, yet sending every task to a distant cloud incurs high transmission latency and energy cost. Mobile Edge Computing (MEC) places computing resources one network hop from the device, creating a three-tier device-edge-cloud system in which each task can be executed locally, at the edge, or in the cloud. This project formulates the per-task choice of tier as a Markov decision process and solves it with tabular Q-learning. The agent learns by interacting with a custom discrete-event simulator built in Python and SimPy, which models Poisson task arrivals, a Shannon-Hartley wireless uplink whose signal quality affects both transmission delay and energy, and a single-server queue at the edge. Each task is scored by a composite cost weighted 0.7 on latency and 0.3 on energy. The agent is compared with five baselines: four fixed strategies (always-local, always-edge, always-cloud and random) and an untrained greedy heuristic that evaluates the known cost model using each task's exact parameters. The simulator's queueing core reproduces M/M/1 theory to within 0.39%, and the complete simulator reproduces closed-form M/G/1 and transmission-delay predictions to within 0.21%. Across five independently trained seeds, evaluated with the learned policy frozen and at a fixed channel quality, Q-learning reduces mean composite cost by 15.60% (95% CI 13.47–17.74%) relative to the strongest fixed baseline, always-cloud (paired $t(4) = 20.31$, $p < 0.001$), and by 19.84% relative to always-edge. It does not match the greedy heuristic: Q-learning's mean cost is 6.32% higher (95% CI 3.57–9.08%; $p = 0.003$). Unlike the agent, the heuristic sees each task's CPU demand. The learned policy never executes locally. It sends a task to the edge when the edge server is idle and to the cloud when the edge is busy, with a few seed-dependent exceptions, mostly when one or two tasks are present. A sweep of the cost weights yields a degenerate latency-energy frontier, because device energy under offloading does not depend on the destination tier. The project delivers an interpretable, reproducible tabular Q-learning baseline and states plainly where it falls short: an untrained, better-informed heuristic outperforms it, and the discrete action space admits no latency-energy trade-off. Both findings motivate the thesis-phase work on richer state representations and continuous control.

Table of Contents

List of Figures.....	
List of Tables	
List of Abbreviations.....	
Chapter 1: Introduction.....	1
1.1 Background.....	1
1.2 Problem Statement.....	1
1.3 Proposed Solution.....	2
1.4 Objectives.....	2
1.5 Scope and Limitations.....	3
1.6 Report Organisation.....	3
Chapter 2: Literature Review.....	4
2.1 Introduction.....	4
2.2 Mobile Edge Computing: Architecture and Motivation.....	4
2.3 Computation Offloading: Problem Formulation.....	5
2.4 Multi-Tier Computing Architectures.....	6
2.5 Traditional and Heuristic Offloading Approaches.....	6
2.6 Reinforcement Learning Approaches to Task Offloading.....	7
2.6.1 Tabular Q-Learning and SARSA (2018-2020).....	7
2.6.2 Deep Q-Networks (2020-2023).....	8
2.6.3 Actor-Critic and DDPG Approaches.....	9
2.6.4 Federated and Multi-Agent DRL (2022-Present).....	9
2.7 Smart City Edge Computing.....	10
2.8 Simulation Tools and Channel Modelling for Edge Computing Research.....	10
2.9 Summary of Research Gaps and Project Positioning.....	12
Chapter 3: Methodology and System Design.....	14
3.1 System Workflow Overview.....	14

3.2 Three-Tier System Model.....	16
3.3 Wireless Channel and Cost Model.....	17
3.4 MDP Formulation.....	18
3.4.1 State Space.....	18
3.4.2 Action Space.....	19
3.4.3 Reward Function.....	19
3.5 Q-Learning Algorithm.....	19
3.6 Simulation Environment.....	21
3.7 Baseline Strategies.....	22
3.8 Experimental Setup.....	23
Chapter 4: Results and Analysis.....	25
4.1 Simulator Validity (M/M/1 and M/G/1 Queueing Theory).....	25
4.2 Performance Comparison.....	26
4.3 Statistical Validation.....	27
4.3.1 Confidence Interval.....	27
4.3.2 Paired t-Test.....	28
4.3.3 Effect Size.....	28
4.4 Learned Policy Analysis.....	30
4.5 Pareto Trade-Off Analysis.....	32
4.6 Sensitivity Analysis.....	33
4.7 Discussion.....	35
Chapter 5: Conclusion and Future Work.....	37
5.1 Summary and Contributions.....	37
5.2 Limitations.....	38
5.3 Future Work (Thesis Phase).....	39
References.....	41

List of Figures

Figure 3.1: End-to-end solution workflow for task generation, state construction, Q-learning, execution, evaluation and validation.

Figure 3.2: Three-tier device-edge-cloud architecture and tier-specific execution cost model.

Figure 3.3: SimPy environment and Q-learning interaction for a single task transition.

Figure 4.1: Average task latency by strategy (five seeds)

Figure 4.2: Q-learning training curves (five seeds)

Figure 4.3: Learned greedy policy per state (seed 42)

Figure 4.4: Latency-energy operating points of seven weightings

List of Tables

Table 3.1: Q-learning and simulation hyperparameters

Table 4.1: SimPy queue versus M/M/1 theory

Table 4.2: Average latency, energy and composite cost by strategy

Table 4.3: Statistical validation summary (five seeds)

Table 4.4: One-at-a-time hyperparameter sensitivity (five seeds)

List of Abbreviations

CI	Confidence Interval
DDPG	Deep Deterministic Policy Gradient
DDQN	Double Deep Q-Network
DQN	Deep Q-Network
DRL	Deep Reinforcement Learning
ETSI	European Telecommunications Standards Institute
FIFO	First-In-First-Out
FL-DDPG	Federated Deep Deterministic Policy Gradient
IoT	Internet of Things
M/G/1	Single-server queue, Poisson arrivals, general service times
M/M/1	Single-server queue, Poisson arrivals, exponential service times
MARL	Multi-Agent Reinforcement Learning
MCC	Mobile Cloud Computing
MDP	Markov Decision Process
MEC	Mobile Edge Computing (also Multi-access Edge Computing)
RL	Reinforcement Learning
SARSA	State-Action-Reward-State-Action
SNR	Signal-to-Noise Ratio

Chapter 1: Introduction

1.1 Background

The number of connected Internet of Things (IoT) devices, smart sensors and mobile platforms has grown rapidly. The applications they run, such as video analytics, augmented reality and real-time environmental monitoring, demand processing power, memory and energy that individual resource-constrained devices cannot sustainably provide. Two architectural responses have historically been used. In Mobile Cloud Computing (MCC), tasks are sent to a powerful but distant cloud data centre, which provides effectively unlimited computing capacity at the cost of high transmission latency and uplink energy. In purely local execution, tasks run on the device itself, avoiding transmission but draining the battery and often failing to meet latency requirements.

Mobile Edge Computing (MEC) introduces an intermediate tier. Edge servers deploy one network hop from the device, for example at base stations or roadside units. An edge server offers more computing power than the device and lower latency and transmission energy than the cloud. The result is a three-tier device-edge-cloud system in which each generated task can be executed locally, offloaded to the edge, or offloaded to the cloud. Choosing the right tier for each task, under changing network conditions, is the computation offloading problem this project addresses.

1.2 Problem Statement

Computation offloading decisions involve a fundamental tension between two conflicting objectives, task completion latency and device energy consumption, under network conditions that vary over time. Traditional offloading policies rely on fixed rules or heuristics, for example thresholds on task size or battery level, or optimisation derived for a fixed, known channel model. These static methods cannot adapt when the bandwidth, server load or signal quality changes. A rule calibrated for light traffic performs poorly at peak load. An optimisation derived for one channel breaks down under fading or interference. The problem is therefore **to learn an adaptive offloading policy** that, observing the current system state, selects the execution tier that minimises a combined latency-energy cost across a wide range of operating conditions.

1.3 Proposed Solution

This project formulates three-tier task offloading as a **Markov Decision Process (MDP)** and solves it with tabular Q-learning, a model-free reinforcement learning algorithm that learns the value of each action in each discretised system state through trial-and-error interaction. The agent is trained and evaluated inside a custom discrete-event simulation written in **Python** and **SimPy**, which models random task arrivals, a realistic wireless channel and edge-server queueing without requiring physical hardware. **Tabular Q-learning** is chosen deliberately for its interpretability. The learned policy is a directly inspectable table, in contrast to the opaque neural networks used in deep reinforcement learning. The agent is benchmarked against five baselines, four fixed strategies and a state-aware but untrained heuristic, to establish how much a learned policy improves upon static strategies and upon an informed non-learning one.

1.4 Objectives

The objectives of this project are:

1. To design and implement a discrete-event simulation of a three-tier device-edge-cloud system in Python and SimPy, modelling task arrivals, wireless transmission, computation and edge queueing.
2. To formulate the task-offloading problem as a Markov Decision Process and implement a tabular Q-learning agent that learns an offloading policy minimising a composite latency-energy cost.
3. To implement baseline strategies for comparison: four fixed strategies (always-local, always-edge, always-cloud and random) and a myopic cost-minimising heuristic that uses the same state information as the agent but requires no training.
4. To validate the simulator against analytical queueing theory and to evaluate, with statistical rigour, whether the Q-learning agent improves upon the best fixed baseline, with a target improvement of at least 10% relative to the always-edge baseline used in the project proposal. A myopic, state-aware heuristic baseline was added during evaluation as a stronger point of comparison than any fixed strategy.

1.5 Scope and Limitations

The scope of this project is a simulation-based study. The contributions are an interpretable, reproducible Q-learning baseline and the simulation environment itself; the project does not claim state-of-the-art performance against deep learning methods. The principal limitations are as follows.

- First, the study uses tabular Q-learning, which requires the system state to be discretised into bins and does not scale to very large state spaces.
- Second, the work is single-agent: it models one device making offloading decisions rather than many devices competing for shared edge resources.
- Third, the cost model makes device energy independent of the offload destination, which removes any latency-energy trade-off from the discrete action space.
- Fourth, all results are obtained in simulation rather than on physical MEC infrastructure. These limitations directly motivate the subsequent thesis phase, which will explore deep and continuous-control reinforcement learning.

Section 5.2 states the full set of limitations precisely.

1.6 Report Organisation

The remainder of this report is organised as follows. Chapter 2 reviews the literature on mobile edge computing, computation offloading and reinforcement learning, and identifies the research gap. Chapter 3 presents the system design and methodology, including the MDP formulation, the Q-learning algorithm and the simulation environment. Chapter 4 reports and analyses the results: simulator validation, performance comparison, statistical validation, learned-policy analysis, the Pareto trade-off, hyperparameter sensitivity and a discussion of what the results do and do not show. Chapter 5 concludes and outlines the future thesis-phase work.

Chapter 2: Literature Review

2.1 Introduction

This chapter reviews the literature relevant to a Q-learning agent that makes task offloading decisions in a three-tier device-edge-cloud system. The review traces the field from foundational Mobile Edge Computing (MEC) concepts and traditional rule-based offloading approaches, through early reinforcement learning methods, to the deep reinforcement learning (DRL) techniques that dominate contemporary research. It concludes by identifying the precise research gap that this study addresses: the absence of a reproducible and interpretable Q-learning baseline for the three-tier terrestrial IoT offloading problem in a Python-based simulation environment.

2.2 Mobile Edge Computing: Architecture and Motivation

Mobile Edge Computing (MEC), initially standardised by the European Telecommunications Standards Institute (ETSI) and later broadened to Multi-access Edge Computing, positions computing, storage and networking resources at the edge of the access network. In the seminal survey by Mao et al. (2017), MEC is defined as a technology that pushes mobile computing, network control and storage to the network edges to enable computation-intensive and latency-critical applications on resource-limited devices. This survey remains a foundational reference, identifying joint radio and computational resource management as the central research challenge and establishing that MEC can deliver substantial reductions in both task execution latency and mobile energy consumption relative to Mobile Cloud Computing.

The case for MEC stems from a fundamental tension. A device that processes tasks locally consumes battery rapidly and may miss latency requirements, whereas a device that sends all tasks to a remote cloud incurs substantial transmission delay and uplink energy, particularly for large payloads over congested networks. Edge servers offer a middle ground, and effective resource management must exploit all three tiers. This relevance extends to smart cities, healthcare IoT, autonomous vehicles and industrial automation, where **workloads have heterogeneous urgency and size profiles, making static offloading rules inadequate.**

2.3 Computation Offloading: Problem Formulation

Computation offloading transfers a task from a resource-constrained device to a more capable remote node. The literature distinguishes binary offloading, in which each task is executed entirely at one location, from partial offloading, in which a task is split across locations at the cost of greater algorithmic complexity (Mao et al., 2017). This study adopts binary offloading across three destinations, local device, edge server or cloud, consistent with comparable studies (Alfakih et al., 2020; Shuai & Xie, 2024) and providing a tractable action space for Q-learning.

The standard optimisation objective in the offloading literature is a weighted sum of task completion latency L and device energy consumption E :

$$C = w_{lat} L + w_{eng} E$$

where L is the task completion latency, E is the device energy consumed, and w_{lat} and w_{eng} are weighting coefficients encoding the application's latency-energy preference, with $w_{lat} + w_{eng} = 1$ (Alfakih et al., 2020; Chen & Liu, 2022). The Q-learning reward in this study is the negative of this cost, because Q-learning maximises cumulative discounted reward while the goal is cost minimisation.

The problem is naturally modelled as a Markov Decision Process (MDP) with state space S , action space A , transition probabilities $P(s' | s, a)$, reward function $r(s, a)$ and discount factor $\gamma \in [0, 1)$ (Sutton & Barto, 2018). The state captures the current system snapshot, including task characteristics, network quality and resource availability; the action is the offloading destination; and an optimal policy maps each state to the action that maximises the expected discounted return, which here is the negative of the expected cumulative discounted cost. The optimal action-value function Q^* satisfies the Bellman optimality equation

$$Q^*(s, a) = \mathbb{E} \left[r + \gamma \max_{a'} Q^*(s', a') \mid s, a \right]$$

and an optimal policy acts greedily with respect to Q^* . When the state space is finite and small enough to enumerate, Q^* can be learned with tabular Q-learning; when it grows

large, neural-network approximation is required. This study addresses the first case, establishing an interpretable tabular Q-learning baseline for a three-tier system.

2.4 Multi-Tier Computing Architectures

Early offloading literature predominantly considered two-tier device-to-cloud or device-to-edge architectures. Although these validated the core offloading concept, they failed to exploit the complementary capabilities of edge and cloud tiers simultaneously, which drove the development of three-tier architectures. In the three-tier model, the device (Tier 1) provides zero-transmission-latency execution but is severely constrained in CPU, memory and battery; the edge server (Tier 2) offers moderate computing capacity at low latency, one hop from the device; and the cloud server (Tier 3) provides near-unlimited computing at the cost of higher transmission latency and energy. A well-designed agent learns to route each task to the tier that minimises the weighted cost under current conditions.

Similar three-tier architectures appear in satellite edge computing (Shuai & Xie, 2024) and vehicular edge computing (Shi et al., 2023). The analytical treatment of three-tier systems using queueing theory provides a mathematical basis for simulation validation. In a three-tier cloud-edge-fog model with probabilistic offloading and Poisson arrivals, the steady-state delay and utilisation can be predicted analytically (Hwang et al., 2021). Comparing simulation outputs against these analytical predictions using the M/M/1 queue benchmark is a standard way of establishing simulator validity, and this validation is a required step in this study.

2.5 Traditional and Heuristic Offloading Approaches

Prior to machine learning, offloading decisions were made using fixed mathematical rules and heuristics, including threshold-based policies, Lyapunov optimisation for online resource allocation under stochastic conditions (Bi et al., 2021), convex optimisation for joint offloading and resource allocation (You et al., 2017), and game-theoretic models for multi-user scenarios (Chen et al., 2016). Mao et al. (2017) provide a comprehensive taxonomy of these methods. Their principal strength is computational efficiency: a threshold rule requires no training, state observation or memory and can run in microcontroller firmware with negligible overhead.

However, these approaches share a critical limitation: they are static. They assume fixed, known network parameters and cannot adapt to the dynamic, time-varying conditions of real deployments. A rule calibrated for morning traffic performs poorly at peak hours; a convex solution derived for a specific channel model breaks down under fading or interference. This rigidity is the core motivation for applying reinforcement learning, which can adapt its policy continuously from experience without an explicit model of the dynamics.

2.6 Reinforcement Learning Approaches to Task Offloading

Reinforcement learning (RL) offers a fundamentally different approach. Rather than encoding fixed rules, an RL agent learns optimal behaviour through trial and error, receiving a scalar reward after each action. The MDP formulation of the offloading problem (Section 2.3) maps directly onto the RL framework. This section traces the evolution of RL methods for offloading from early tabular approaches to the DRL techniques that dominate current research.

2.6.1 Tabular Q-Learning and SARSA (2018-2020)

Q-learning, introduced by Watkins (1989) and proved convergent by Watkins and Dayan (1992), is a model-free, off-policy algorithm that learns a table of action-value estimates $Q(s, a)$ by repeatedly applying a sampled form of the Bellman optimality equation. The first wave of RL-based offloading papers (2018-2020) applied Q-learning and its variants to show that adaptive policies outperform static heuristics under dynamic network conditions. Alfakih et al. (2020) proposed a SARSA-based algorithm, an on-policy variant, for MEC resource management, minimising system cost (energy and delay) in a multi-server, multi-user architecture with three offloading options: the nearest edge server, an adjacent edge server and the remote cloud. This multi-tier framing anticipates the three-tier hierarchy of this study and represents one of the first systematic RL treatments of multi-destination offloading, although its multi-server architecture introduces complexity that limits scalability.

Shuai and Xie (2024) provide the most structurally comparable work to this study. Their Q-learning-based offloading strategy operates on a three-tier framework of collaborative edge devices, edge data centres and cloud data centres in a satellite edge computing

environment. It optimises communication latency, energy consumption and task success rate, and outperforms baseline algorithms across all three metrics. However, the satellite context introduces domain-specific constraints, such as orbital mechanics, long propagation delays and variable coverage, that are absent in terrestrial IoT and smart-city deployments. In the literature reviewed for this project, no equivalent Q-learning baseline was found for the general-purpose device-edge-cloud architecture in terrestrial scenarios; developing such a baseline is the primary contribution of this study. More recently, Dash et al. (2026) proposed a modified Q-learning approach for joint task offloading and resource allocation in MEC, confirming that tabular methods remain of active research interest; their work, however, targets resource allocation within a two-tier setting rather than three-tier destination selection.

The fundamental limitation of tabular Q-learning is scalability: the Q-table must enumerate every state-action pair, so its size grows exponentially with the number of state variables. For a system with a modest number of discrete state variables this is manageable and the policy remains fully interpretable; for large-scale multi-user, multi-server scenarios the table becomes intractable and neural-network approximation is necessary.

2.6.2 Deep Q-Networks (2020-2023)

The Deep Q-Network (DQN), introduced by Mnih et al. (2015), replaces the Q-table with a neural network that approximates $Q(s, a; \theta)$, enabling generalisation across unseen states. Two innovations stabilise DQN training: an experience replay buffer that breaks temporal correlations between samples, and a target network that provides stable training targets. These were subsequently imported into MEC offloading research. The Double Deep Q-Network (DDQN) improves on DQN by decoupling action selection from value estimation, reducing the overestimation bias that can destabilise training. Sun et al. (2026) apply a DDQN with an action mask that prevents the agent from selecting infeasible actions to a three-tier cloud-edge-device architecture, jointly optimising delay, energy, mobility and economic cost; this illustrates the direction the field has taken and the performance ceiling against which an interpretable tabular baseline must be positioned.

The transition from Q-learning to DQN is not cost-free. A Q-table is fully transparent: an operator can inspect any row and see the value assigned to each action in that state. A neural

network is an opaque function approximator whose internal representations are not directly interpretable. In smart-city deployments, where operators must audit, justify and override automated decisions, interpretability is a practical operational requirement, not merely an academic concern. This motivates the need for a Q-learning baseline even in an era dominated by DRL.

2.6.3 Actor-Critic and DDPG Approaches

The Deep Deterministic Policy Gradient (DDPG), introduced by Lillicrap et al. (2016), extends DRL to continuous action spaces using an actor that maps states to actions and a critic that evaluates state-action pairs. This enables joint optimisation of discrete offloading decisions and continuous resource-allocation variables such as CPU frequency and transmission power, making DDPG a powerful tool for the full joint optimisation problem.

Chen and Liu (2022) apply Federated DDPG (FL-DDPG) to smart-city MEC, combining federated learning with DDPG to address privacy. In their system, IoT devices offload tasks to MEC servers or the central cloud subject to resource and latency constraints; federated learning shares model gradient updates rather than raw data, improving both training performance and privacy compared with vanilla DDPG. It is important to note that federated learning in this sense has a specific technical meaning: multiple agents share model updates privately without exchanging raw data. This is categorically different from merely distributing computation across tiers. Because the present study uses a single agent and does not implement cross-agent parameter sharing, it adopts the three-tier device-edge-cloud terminology rather than the federated label; federated multi-agent learning is identified as future work (Section 5.3).

2.6.4 Federated and Multi-Agent DRL (2022-Present)

The most recent phase of the field, from approximately 2022 to the present, is characterised by multi-agent DRL, federated approaches and integration with 6G network architectures. Survey evidence (Peng et al., 2024) confirms this trajectory, identifying multi-agent coordination, 6G-enabled edge networks and meta-learning as dominant themes. Federated DRL approaches, in which distributed edge nodes collaboratively train a shared model without sharing raw data, address privacy in sensitive scenarios such as healthcare IoT, at the cost of communication overhead and architectural complexity. Multi-agent DRL

introduces game-theoretic dynamics, in which the optimal policy for one agent depends on the policies of others, capturing the competitive nature of multi-user MEC but substantially complicating training and analysis. The explosion of publications in this space is precisely why a simple, interpretable baseline is needed: complex multi-agent DRL results are difficult to compare without a well-characterised reference point.

2.7 Smart City Edge Computing

Smart cities provide the primary application context for this study. Smart-city infrastructure integrates diverse IoT endpoints, including traffic sensors, environmental monitors, surveillance cameras and industrial automation devices, each generating latency-sensitive and computation-intensive workloads with heterogeneous characteristics. Mobile edge servers positioned throughout the urban area act as Tier 2 nodes, while a central cloud data centre serves as Tier 3. Chen and Liu (2022) address this context directly, reporting substantial performance improvements over fixed baselines and vanilla DDPG in a multi-user smart-city scenario, though the complexity of FL-DDPG limits its interpretability.

The smart-city context also motivates the energy dimension. IoT devices deployed in urban infrastructure are often battery-powered or energy-harvesting, so minimising energy extends operational lifetime and reduces maintenance. The latency-energy trade-off, controlled by the weighting parameters w_{lat} and w_{eng} , is therefore a practical design parameter. This study sweeps these weights and reports the result as discussed in Chapter 4. The frontier proves to be degenerate under discrete three-action offloading, for the structural reason analysed in Section 4.5, which is itself an informative result about the limits of discrete-action policies.

2.8 Simulation Tools and Channel Modelling for Edge Computing Research

Reproducible simulation is essential for validating offloading algorithms before deployment, since direct experimentation on live MEC infrastructure is expensive, unrepeatable and difficult to control. The dominant simulation tool in MEC offloading literature is EdgeCloudSim (Sonmez et al., 2018), a Java-based platform built on CloudSim that models mobile applications, edge servers, network conditions and device mobility. Its maturity makes it a reference, but its Java implementation creates a language barrier for the

machine-learning research community, which predominantly works in Python. Souza et al. (2023) address this gap with EdgeSimPy, a Python-based edge computing simulation framework built on discrete-event principles; however, EdgeSimPy focuses on resource provisioning and service migration rather than on RL-driven three-tier offloading with explicit latency-energy cost functions.

This study therefore uses a custom Python/SimPy simulation tailored to the three-tier offloading decision problem. The simulator models task arrivals as a Poisson process, tracks processing queues at the edge, and, importantly, computes the wireless uplink data rate using the Shannon-Hartley theorem, $R = B \log_2(1 + \text{SNR})$, where the signal-to-noise ratio (SNR) is derived from the current network-quality state. This makes network quality a variable that genuinely affects cost: a weak signal yields a lower data rate, so the same task takes longer and consumes more energy to transmit. The validity of the simulator is established through comparison against M/M/1 queueing-theory predictions for individual server stages, following the analytical three-tier queueing treatment of Hwang et al. (2021).

2.9 Summary of Research Gaps and Project Positioning

The preceding review identifies four interconnected gaps that motivate this study.

First, the literature reviewed contains no reproducible Q-learning baseline for the general-purpose terrestrial three-tier device-edge-cloud offloading problem. The most comparable work (Shuai & Xie, 2024) applies Q-learning in a satellite-specific context with domain constraints absent from smart-city IoT, while Alfakih et al. (2020) use SARSA in an effectively two-tier architecture. In the work reviewed, the field moved directly from early tabular methods to DQN and DDPG without establishing an interpretable tabular baseline for the three-tier terrestrial case.

Second, existing simulation tools for MEC offloading are predominantly Java-based (EdgeCloudSim) or focused on resource provisioning (EdgeSimPy). No published work was identified that implements a Python/SimPy simulation specifically for RL-driven three-tier offloading with explicit latency-energy cost optimisation; a Python-based simulator reduces barriers to reproduction and extension by the machine-learning community.

Third, the literature lacks a study that characterises the performance ceiling of Q-learning in a three-tier system and uses those measurements to formally motivate the transition to DRL. Survey work (Peng et al., 2024) describes the historical evolution, but no study instantiates it as a controlled experiment. This gap is not closed by the present project; it defines the thesis phase (Section 5.3).

Fourth, existing smart-city three-tier offloading work (Chen & Liu, 2022) uses opaque DRL models whose decision-making is not directly inspectable. An interpretable Q-table approach, in which every state-action value is directly readable, provides the transparency operators need to audit and override decisions.

This study is therefore positioned as a clean, reproducible, interpretable Q-learning baseline for three-tier device-edge-cloud task offloading in a Python/SimPy simulation. The contribution is framed not as claiming state-of-the-art performance, but as providing the missing baseline that makes meaningful comparison possible and that explicitly motivates DRL as the necessary next step.

Chapter 3: Methodology and System Design

3.1 System Workflow Overview

Figure 3.1 summarises the end-to-end workflow of the project, from the raw data source through to the final evaluated outcome. Tasks originate from the SimPy discrete-event simulator (Section 3.6), which draws Poisson task arrivals, sizes, computational complexity and network quality. These raw values are converted into a discrete MDP state (Section 3.4.1). The tabular Q-learning agent (Section 3.5) is trained on this stream of states using an ϵ -greedy policy over 2,000 episodes. Two independent checks are then applied before any performance claim is made: the simulator is checked against closed-form queueing and transmission results (Section 4.1), and the trained agent, with learning switched off, is evaluated greedily against the four fixed baselines and the Greedy Heuristic across five independently seeded runs with disjoint training data, with paired t-tests establishing statistical significance (Section 4.3). The outcome is a learned offloading policy together with the cost-reduction, sensitivity and Pareto-sweep results reported in Chapter 4.

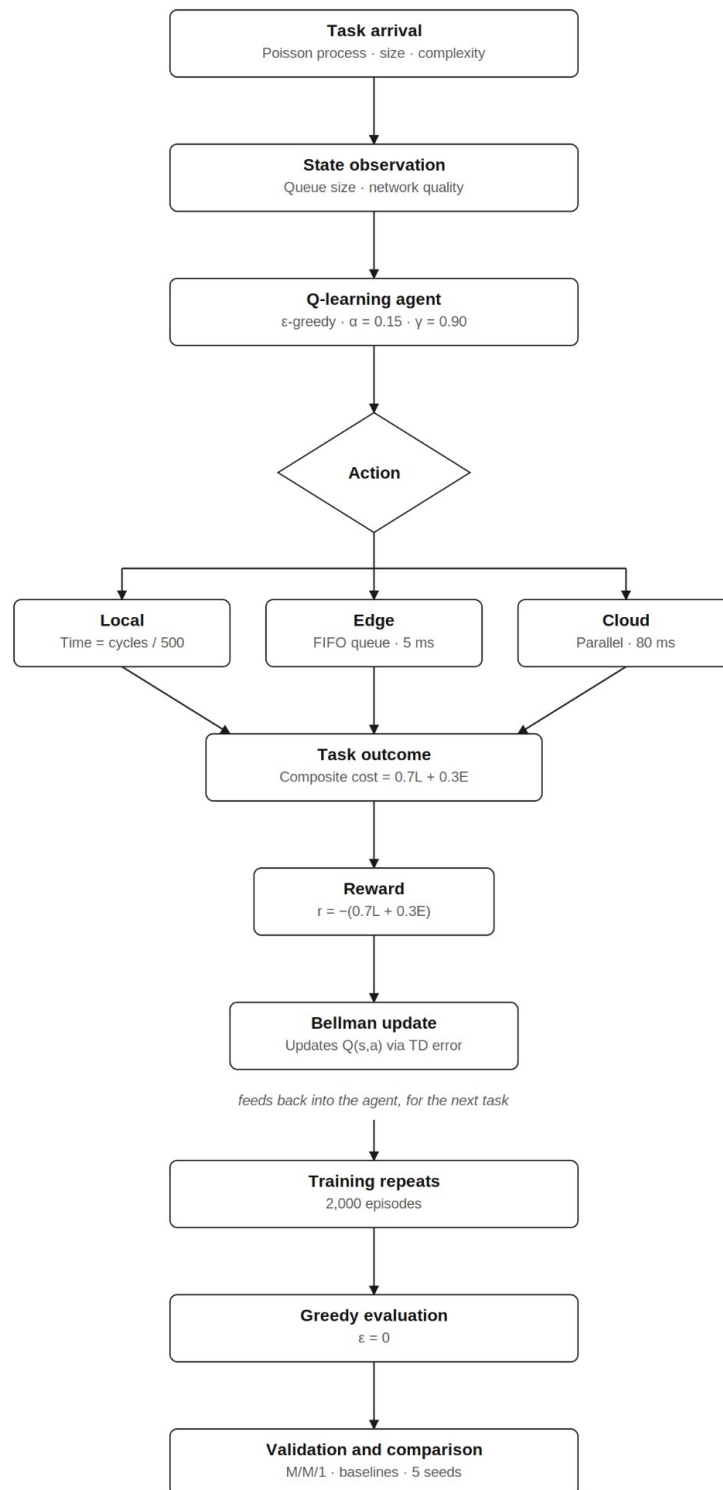


Figure 3.1: End-to-end solution workflow for task generation, state construction, Q-learning, execution, evaluation and validation. The observed state also includes the task-size bin (Section 3.4.1); evaluation is greedy with learning switched off.

3.2 Three-Tier System Model

The system comprises three computing tiers. Tier 1 is the local device, with a modest processor (modelled at 500 million CPU cycles per second) and a fixed compute power draw of 0.5 W. Local execution incurs no transmission cost but is slow for demanding tasks. Tier 2 is the edge server, with substantially higher processing speed (8,000 million cycles per second), a short propagation delay (5 ms) and a single-server First-In-First-Out queue. The queue is unbounded: no task is ever rejected, and at the arrival rate studied ($\lambda = 6$ tasks/s against an effective service rate of approximately 14.5 tasks/s, utilisation $\rho \approx 0.41$ under Always-Edge) the queue is stable, so a capacity bound would not bind. Tier 3 is the cloud server, with the highest processing speed (30,000 million cycles per second) but the longest propagation delay (80 ms) and effectively unlimited capacity. Tasks are generated at the device and must be assigned to exactly one tier. Figure 3.2 summarises the architecture and the per-tier cost model.

Implemented Execution Architecture

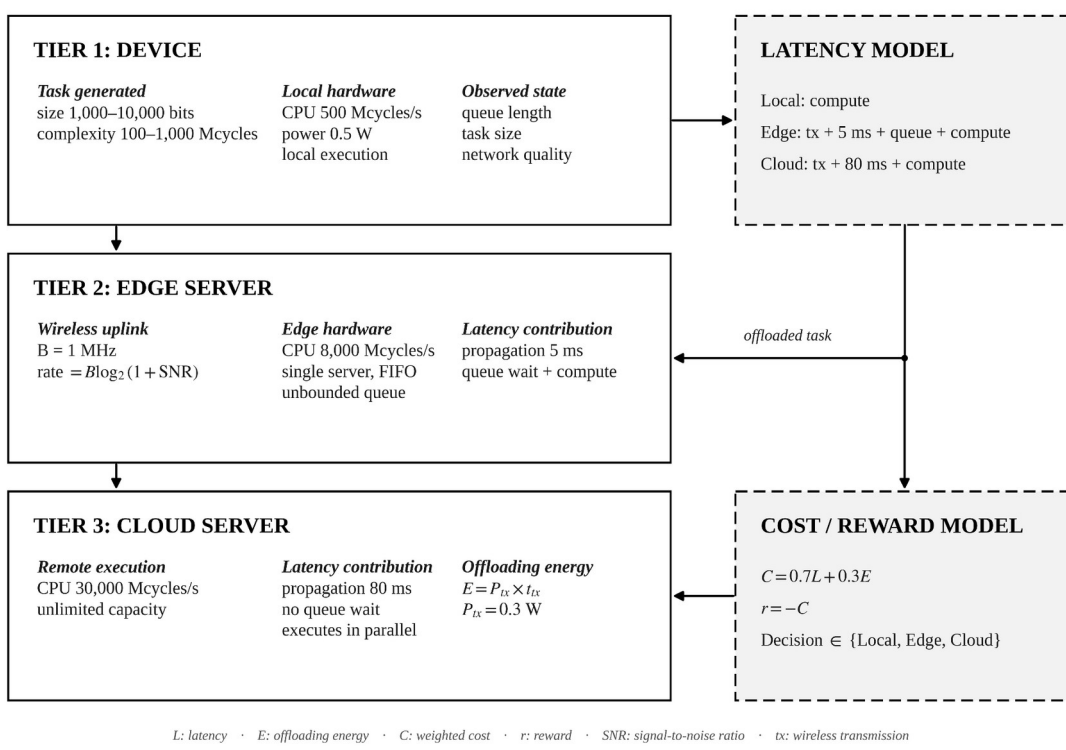


Figure 3.2: Three-tier device-edge-cloud architecture and tier-specific execution cost model.

3.3 Wireless Channel and Cost Model

Offloading to the edge or cloud requires transmitting the task over a wireless uplink. The achievable data rate R is computed using the Shannon-Hartley theorem:

$$R = B \log_2(1 + \text{SNR})$$

where B is the channel bandwidth (1 MHz) and the signal-to-noise ratio (SNR) is mapped from the current network-quality value on a decibel scale ranging from -10 dB (poor signal, approximately 137 kbit/s) to 30 dB (excellent signal, approximately 10 Mbit/s). The transmission delay of a task is its size in bits divided by this rate, and the transmission energy is the transmission power (0.3 W) multiplied by the transmission delay. This design ensures that network quality genuinely affects both latency and energy, rather than being an inert state variable. The total latency and energy for each tier are:

- Local: latency and energy from computation only (no transmission).
- Edge: transmission delay + propagation (5 ms) + queue wait + computation, with transmission energy.
- Cloud: transmission delay + propagation (80 ms) + computation, with transmission energy.

In the simulator, an offloaded task joins the edge queue at the instant it is generated; its transmission and propagation delays are added to its latency analytically rather than being advanced in simulated time. At the evaluation channel these delays (about 0.6 ms of transmission and 5 ms of propagation) are small compared with the mean edge service time of 69 ms. As a robustness check, all five seeds were re-run with both delays simulated in event time (the task joins the edge queue only after its upload): the improvement over Always-Cloud becomes 15.79% (95% CI [15.34%, 16.23%]) and the difference to the Greedy Heuristic -4.89% (95% CI $[-5.13\%, -4.64\%]$), so the conclusions do not depend on this simplification.

Each task is scored by a composite cost combining latency and energy with weights $w_{\text{lat}} = 0.7$ and $w_{\text{eng}} = 0.3$, where L is the task latency in seconds and E the device energy in joules, reflecting a latency-sensitive application:

$$C = 0.7L + 0.3E$$

3.4 MDP Formulation

The offloading problem is formulated as a Markov Decision Process. The state, action and reward are defined as follows.

3.4.1 State Space

The agent observes a discretised state. In the base configuration the state is a tuple of three binned variables: the number of tasks at the edge server, counting both waiting tasks and the task in service (4 bins: 0, 1–2, 3–5 and 6 or more), the task size (3 bins, with boundaries at 3,000 and 7,000 bits) and the network quality (3 bins, with boundaries at 0.4 and 0.75). This

yields $4 \times 3 \times 3 = 36$ nominal states, of which the 27 with fewer than six tasks at the edge are reached in practice at the arrival rate studied; Section 4.4 reports the coverage actually achieved. The state deliberately omits the task's CPU demand (complexity), which strongly affects computation time; from the agent's perspective it is unobserved noise. A mobility extension adds device velocity (3 bins) and a network-quality trend (3 bins), expanding the nominal state space to $4 \times 3 \times 3 \times 3 \times 3 = 324$ states.

3.4.2 Action Space

The action space has three discrete actions: execute locally (0), offload to the edge (1), or offload to the cloud (2).

3.4.3 Reward Function

Because Q-learning maximises the expected discounted sum of rewards while the objective is cost minimisation, the reward for a task is the negative of its composite cost:

$$r = -(0.7L + 0.3E)$$

A decision that yields low latency and energy therefore receives a reward close to zero (the best achievable), while an expensive decision receives a large negative reward.

3.5 Q-Learning Algorithm

The agent maintains a Q-table mapping each state to a vector of action-value estimates, one per action, initialised to zero. Because every reward in this problem is strictly negative, a zero initialisation is optimistic: an action that has never been tried in a given state retains a value of zero, which exceeds the value of every action that has been tried. During training this is desirable, since it drives systematic exploration of untried actions. At greedy evaluation time, however, it would allow an untried action to win the arg-max and be reported as a learned decision. The implementation therefore records a visit count $N(s, a)$ for every state-action pair and restricts every arg-max and max to actions that have actually been tried; states never visited during training are reported as untrained rather than being assigned a spurious preference. When a task completes, the agent updates the value of the action chosen for it:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a': N(s', a') > 0} Q(s', a') - Q(s, a) \right]$$

where α is the learning rate, γ the discount factor, r the task's reward and s' the state observed when the task completes; the maximum is 0 if no action has yet been tried in s' . The task stream has no terminal state, and the 100-second episode boundary is a time limit rather than a termination, so every update bootstraps from s' . Because tasks overlap in time, updates are applied asynchronously in order of task completion. Action selection is ϵ -greedy: with probability ϵ the agent picks a uniformly random tier, and otherwise the tried action with the highest Q-value. ϵ starts at 1.0 and is multiplied by 0.998 after each episode down to a floor of 0.05, which it reaches after 1,497 episodes. If a state was never visited in training, the agent falls back to the edge; such states arise only when six or more tasks are at the edge. After training the agent is frozen: ϵ is set to zero and the Q-table is no longer updated, so evaluation measures exactly the policy that was learned. Algorithm 1 summarises the procedure as implemented, and Table 3.1 lists the hyperparameters.

Algorithm 1: Tabular Q-learning for task offloading
 (implementation: src/agent/q_learning_agent.py)
 Input: $\alpha = 0.15$, $\gamma = 0.90$, $\epsilon: 1.0 \rightarrow 0.05$, 2,000 episodes
 1: $Q(s,a) \leftarrow 0$, $N(s,a) \leftarrow 0$ for all s, a $\triangleright$ optimistic (rewards < 0)
 2: for episode = 1, ..., 2,000 do
 3: $q \sim U(0.1, 1.0)$; simulate 100 s of Poisson task arrivals
 4: for each arriving task do
 5: $s \leftarrow (\text{edge-load bin, task-size bin, channel-quality bin})$
 6: $a \leftarrow \text{random tier w.p. } \epsilon, \text{ else } \text{argmax}_{\{a: N(s,a)>0\}} Q(s,a)$
 7: run task on tier a ; on completion get $r = -(0.7L + 0.3E)$, s'
 8: $y \leftarrow r + \gamma \cdot \max_{\{a': N(s',a')>0\}} Q(s',a')$ ($y \leftarrow r$ if none)
 9: $Q(s,a) \leftarrow Q(s,a) + \alpha (y - Q(s,a))$; $N(s,a) \leftarrow N(s,a) + 1$
 10: $\epsilon \leftarrow \max(0.05, 0.998 \cdot \epsilon)$
 11: freeze ($\epsilon \leftarrow 0$, no updates); evaluate greedily on new workloads

Table 3.1: Q-learning and simulation hyperparameters (src/config.py; the single configuration used for every reported result).

Parameter	Symbol	Value
Learning rate	α	0.15
Discount factor	γ	0.90
Initial exploration	ϵ_0	1.0
Final exploration	$\epsilon_{\min}$	0.05
Exploration decay	—	$\times 0.998$ per episode (floor reached at episode 1,497)
Training episodes	—	2,000

Latency weight	w_{lat}	0.7
Energy weight	w_{eng}	0.3
Task arrival rate	λ	6 tasks/s (Poisson)
Channel bandwidth	B	1 MHz
Q-table initialisation	Q_0	0 (optimistic; rewards are negative)
Episode length	–	100 simulated seconds (~600 tasks)
Training channel quality	q	$U(0.1, 1.0)$, redrawn each episode
Evaluation channel quality	q	0.9, fixed
Evaluation length	–	1,000 s per seed (Table 4.3); 10,000 s (Table 4.2)
Seeds	–	42, 123, 456, 789, 999 (disjoint simulation-seed blocks)

3.6 Simulation Environment

The environment is a custom discrete-event simulation built with SimPy. Tasks arrive following a Poisson process at an average rate of 6 per second; each task has a randomly drawn size (1,000-10,000 bits) and computational complexity (100-1,000 million CPU cycles). Each training episode simulates 100 seconds of system time, yielding approximately 600 tasks and therefore approximately 600 Q-learning updates per episode, or approximately 1.2 million updates over the full 2,000-episode training run. Network quality is held constant within an episode and re-drawn from a uniform distribution over the full range $[0.1, 1.0]$ at the start of each episode, so that every network-quality bin, including the poor-signal bin, is visited during training. The simulation advances from event to event (task arrival, task completion) rather than in fixed time steps, faithfully representing queueing dynamics at the edge server. Figure 3.3 shows the interaction between the simulator and the agent for a single task. The use of simulation means experiments are fully

reproducible, cost nothing to run and allow controlled parameter sweeps that would be impossible on physical hardware.

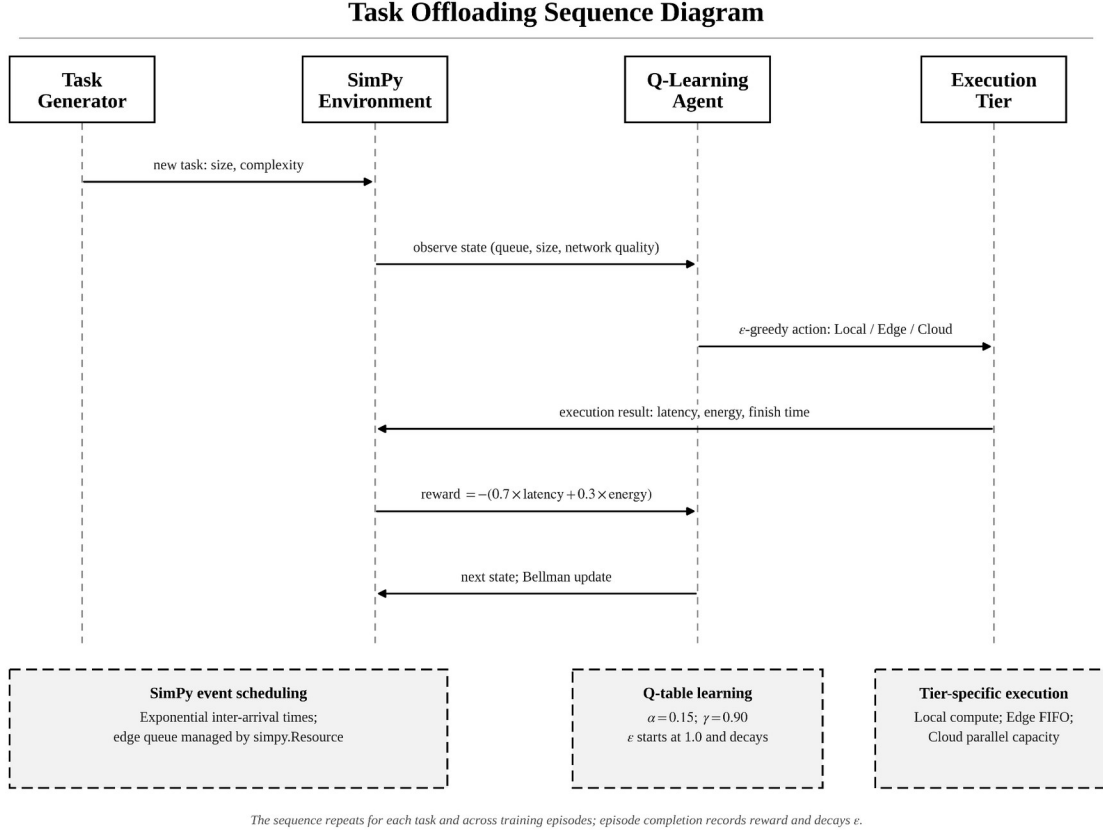


Figure 3.3: SimPy environment and Q-learning interaction for a single task transition.

3.7 Baseline Strategies

To quantify the benefit of learning, the Q-learning agent is compared against five baseline strategies. Four are fixed and state-blind: Always-Local (every task executed on the device), Always-Edge (every task offloaded to the edge), Always-Cloud (every task offloaded to the cloud) and Random (each task assigned to a uniformly random tier). The fifth, the Greedy Heuristic, is state-aware but untrained: for each arriving task it evaluates the same cost model the reward function uses, once per candidate tier, from the task's exact size and CPU demand, the exact current network quality and an estimate of the edge queue wait (the number of tasks at the edge multiplied by the mean service time), and selects the tier with the lowest estimated cost. It holds no Q-table and does not improve with experience. The comparison is deliberately asymmetric in the heuristic's favour. The

heuristic observes raw, undiscretised quantities, including the task's CPU demand, which is not part of the agent's state at all, and it is given the cost model in closed form. The agent sees three binned variables and must learn the consequences of its actions from rewards alone. Beating the four fixed strategies shows only that adapting to state helps. The Greedy Heuristic is a strong, information-advantaged reference: it shows what a myopic rule with full task information achieves, rather than isolating the value of learning alone.

3.8 Experimental Setup

Each Q-learning agent is trained for 2,000 episodes of 100 simulated seconds each, with network quality re-drawn per episode from the full range [0.1, 1.0]. It is then frozen and evaluated greedily: exploration is switched off ($\epsilon = 0$) and the Q-table is no longer updated, so the evaluated policy is exactly the policy learned in training. All greedy evaluations use a single fixed, documented channel condition, network quality 0.9, so that the headline comparison, the five-seed statistics, the sensitivity analysis and the Pareto sweep are all measured under identical conditions. The evaluation workload is drawn from a simulation seed outside every training range, so it is disjoint from every episode the agent trained on. To ensure results are not an artefact of a single random seed, the entire train-and-evaluate procedure is repeated for five seeds (42, 123, 456, 789, 999). Each seed trains on its own block of ten thousand simulation seeds ($\text{seed} \times 10,000 + \text{episode number}$), so the five training runs draw from non-overlapping simulated workloads. For each seed, the agent and all five baselines are evaluated on the same 1,000-second evaluation workload (about 6,000 tasks). Always-Cloud is the cheapest fixed baseline on every seed, so the headline improvement is measured against it, and separately against the Greedy Heuristic. Performance is measured by average task latency, average device energy and the composite cost. Following the reporting practice recommended by Henderson et al. (2018), results are given as means over seeds with 95% confidence intervals (Student t, four degrees of freedom), differences are tested with paired t-tests, p-values are adjusted for the five comparisons with the Holm-Bonferroni method, and hyperparameter sensitivity is reported (Section 4.6). The simulator is validated against closed-form results before any performance claim is made (Section 4.1). An audit of the first version of this study found that the Q-table was still being updated during the greedy evaluation runs. Every evaluation

result in this report was regenerated with the frozen policy; training, and therefore the learned Q-tables, coverage and convergence figures, was unaffected.

Chapter 4: Results and Analysis

4.1 Simulator Validity (M/M/1 and M/G/1 Queueing Theory)

Before drawing conclusions from the simulator, it was checked against closed-form results in two ways. First, the SimPy single-server first-in-first-out resource that implements the edge queue was configured, in a standalone script, as an M/M/1 system (Poisson arrivals, exponential service, one server), and its measured mean time in system was compared with the closed-form prediction $W = 1 / (\mu - \lambda)$. With the service rate fixed at $\mu = 14.5$ tasks per second, the arrival rate was swept across four load levels from light to heavy utilisation (Table 4.1).

Table 4.1: SimPy queue versus M/M/1 theory (standalone SimPy queue, service rate $\mu = 14.5/s$).

Arrival rate λ (1/s)	Utilisation ρ	W theory (s)	W simulated (s)	Verdict
3.0	0.21	0.0870	0.0870	PASS
6.0	0.41	0.1176	0.1176	PASS
9.0	0.62	0.1818	0.1811	PASS
12.0	0.83	0.4000	0.3988	PASS

All four load levels matched the analytical prediction, with a maximum relative error of 0.39% (at $\rho = 0.62$) against a 5% acceptance tolerance. The simulated values are means over between 150,428 and 600,524 served tasks per load level. This check validates the queueing primitive, not the full simulator, whose edge service times are uniformly distributed rather than exponential. Second, therefore, the complete simulator was run under the fixed policies, for which the expected latency has a closed form at the evaluation channel (quality 0.9; five 10,000-second runs each). Under Always-Edge the edge is an M/G/1 queue, and the Pollaczek-Khinchine mean wait plus transmission, propagation and mean service time predicts 0.1039 s against 0.1037 s simulated (0.21% error). Under Always-Cloud the prediction is 0.0990 s against 0.0990 s (0.001%). Under Always-Local it is 1.1000 s against 1.0999 s (0.01%). The simulator is therefore consistent with queueing theory at the level of both its components and its end-to-end latency. Both checks are reproducible (python main.py --agent validate; python -m pytest tests).

4.2 Performance Comparison

Table 4.2 reports the average task latency, average device energy and resulting composite cost for each strategy on the long single-seed evaluation run (seed 42, network quality 0.9, 10,000 simulated seconds, approximately 59,940 tasks per strategy; slower strategies complete marginally fewer tasks within the fixed window). The Greedy Heuristic achieves the lowest latency and the lowest composite cost of all six strategies. The Q-learning agent is second, 14.18% cheaper than Always-Cloud and 8.32% more expensive than the heuristic on this seed. Always-Cloud is the strongest fixed baseline, narrowly ahead of Always-Edge, because the realistic Shannon channel makes the cloud's superior processing speed worthwhile despite its longer propagation delay. Always-Local is by far the worst, confirming that offloading is essential for this workload. The four strategies that never execute locally (Q-learning, Greedy Heuristic, Always-Cloud and Always-Edge) have the same average energy. This follows from the cost model, as Section 4.5 explains. Figure 4.1 shows the average latency of each strategy over all five seeds.

Table 4.2: Average latency, energy and composite cost by strategy ($\text{cost} = 0.7 \times \text{latency} + 0.3 \times \text{energy}$; lower is better). Single representative seed (42), 10,000-second evaluation with the frozen policy; five-seed statistics are given in Table 4.3.

Strategy	Average latency (s)	Average energy (J)	Composite cost	Note
Greedy Heuristic	0.0784	0.000195	0.0550	Best overall (untrained)
Q-learning	0.0850	0.000195	0.0595	Learned policy (this work)
Always-Cloud	0.0990	0.000195	0.0694	Best fixed baseline
Always-Edge	0.1038	0.000195	0.0727	
Random	0.4290	0.1846	0.3557	
Always-Local	1.1016	0.5508	0.9364	Worst

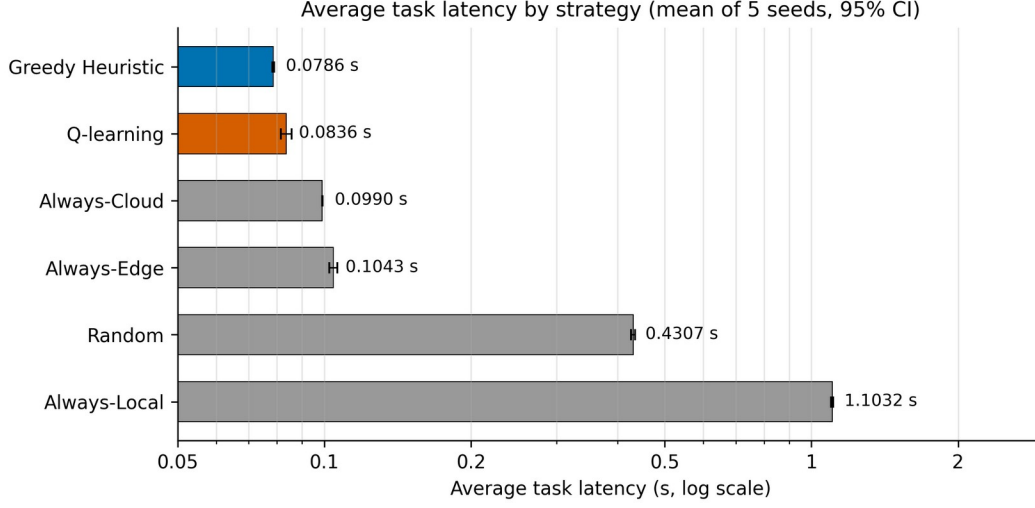


Figure 4.1: Average task latency by strategy, as the mean over five seeds (1,000-second evaluation each; logarithmic axis; error bars show 95% confidence intervals). Q-learning (vermillion) is the learned policy under study; the Greedy Heuristic (blue) achieves the lowest latency, and Always-Cloud is the strongest fixed baseline.

4.3 Statistical Validation

Because a single run could beat a baseline by chance, the comparison was repeated across five seeds and subjected to formal statistical tests. Each seed trains on its own disjoint block of simulation seeds and is evaluated on its own disjoint workload, so the five runs are independent replicates that differ in training workload, exploration noise and evaluation workload. The main comparisons are against Always-Cloud, the strongest fixed baseline, and against the Greedy Heuristic, which is given the cost model and the raw task parameters but requires no training; Table 4.3 also reports Always-Edge, Random and Always-Local. Three analyses were conducted for each comparison.

4.3.1 Confidence Interval

The 95% confidence interval gives the range of plausible values for the mean improvement. It is computed as the mean of the five per-seed improvements plus or minus the Student-t critical value ($t = 2.776$ for four degrees of freedom) times the standard error; the large-sample value of 1.96 would understate the width for five samples. Relative to Always-Cloud, the mean improvement is 15.60%, with a 95% confidence interval of [13.47%, 17.74%] (standard deviation 1.72 percentage points); the per-seed improvements range

from 13.59% to 17.75%. Relative to Always-Edge, the baseline used to set the proposal's 10% target, the mean improvement is 19.84% [16.68%, 22.99%], so the target is met with the whole interval above it. Relative to the Greedy Heuristic, however, the mean difference is -6.32% [-9.08%, -3.57%]; the per-seed values range from -8.81% to -3.95%. The entire interval lies below zero, so the agent is more expensive than the heuristic on every seed, not merely on average.

4.3.2 Paired t-Test

The paired t-test asks whether a mean difference could plausibly be zero. For each seed there is a matched pair, the Q-learning cost and the reference cost measured on the identical evaluation workload, and the test examines the five per-seed differences in composite cost. The null hypothesis is that the mean difference is zero. The t-statistic is the mean difference divided by its standard error. Pairing cancels seed-to-seed variation in the workload and therefore increases sensitivity. Against Always-Cloud the result is $t(4) = 20.31$, $p = 3.5 \times 10^{-5}$. Against the Greedy Heuristic it is $t(4) = -6.38$, $p = 3.1 \times 10^{-3}$; the negative sign reflects the direction of the difference. After a Holm-Bonferroni correction for the five comparisons in Table 4.3, the adjusted p-values are 1.0×10^{-4} and 3.1×10^{-3} , both below 0.05. The agent is therefore significantly cheaper than Always-Cloud and significantly more expensive than the Greedy Heuristic. With five seeds the test has limited power and assumes approximately normal per-seed differences, so the confidence intervals above are the more informative summary. Both statistics were cross-checked against the SciPy implementation and agree to machine precision.

4.3.3 Effect Size

Cohen's d_z for paired data measures the size of an effect relative to its variability: the mean difference divided by the standard deviation of the differences. Against Always-Cloud, $d_z = 9.08$; against the Greedy Heuristic, $d_z = -2.85$. Both far exceed the conventional 0.8 threshold for a large effect. That is because the per-seed differences are consistent relative to their spread, not because either percentage is unusually large; d_z describes consistency across seeds, and the percentages above describe practical size. One methodological note is worth recording. The first version of this analysis reported $17.03\% \pm 0.22\%$ and $d_z = 101.05$ against Always-Cloud, and -4.53% against the heuristic. Those evaluation runs still

updated the Q-table, so each agent went on adapting to the evaluation channel while it was being measured. With the policy frozen, the five independently trained agents keep their differences in the few near-tied states described in Section 4.4. The confidence interval therefore widens, d_z falls and the gap to the heuristic grows. The frozen-policy figures reported here are the correct measure of what was learned in training.

Table 4.3: Statistical validation summary (five seeds, 1,000-second frozen greedy evaluation per seed; costs as mean $\pm$ standard deviation across seeds; improvement = (baseline – Q-learning) / baseline).

Quantity	Value
Q-learning composite cost	0.05856 ± 0.00120
Greedy Heuristic composite cost	0.05508 ± 0.00021
Always-Cloud composite cost	0.06939 ± 0.00006
Always-Edge composite cost	0.07308 ± 0.00112
Random composite cost	0.35711 ± 0.00277
Always-Local composite cost	0.93775 ± 0.00415
Improvement vs Always-Cloud	15.60%, 95% CI [13.47%, 17.74%]; $t(4) = 20.31$; $p = 3.5 \times 10^{-5}$ (Holm 1.0×10^{-4}); $d = 9.08$
Improvement vs Always-Edge	19.84%, 95% CI [16.68%, 22.99%]; $t(4) = 15.71$; $p = 9.6 \times 10^{-5}$ (Holm 1.9×10^{-4}); $d = 7.02$
Improvement vs Greedy Heuristic	–6.32%, 95% CI [–9.08%, –3.57%]; $t(4) = -6.38$; $p = 3.1 \times 10^{-3}$ (Holm 3.1×10^{-3}); $d = -2.85$
Improvement vs Random	83.60%, 95% CI [83.11%, 84.09%]; $t(4) = 200.44$; $p = 3.7 \times 10^{-9}$ (Holm 1.5×10^{-8}); $d_z = 89.64$
Improvement vs Always-Local	93.75%, 95% CI [93.60%, 93.91%]; $t(4) = 472.15$; $p = 1.2 \times 10^{-10}$ (Holm 6.0×10^{-10}); $d_z = 211.15$
Verdict	Significantly cheaper than every fixed

baseline; significantly more expensive than
the Greedy Heuristic

4.4 Learned Policy Analysis

During evaluation at the fixed channel quality of 0.9, the learned policy of seed 42 routes 78.5% of tasks to the edge and 21.5% to the cloud (five-seed range 70.3–78.2% to the edge in the 1,000-second evaluations) and never executes a task locally. The policy is essentially a threshold on edge load. When the edge server is idle, the edge is selected because it offers the lowest latency (44 of the 45 idle-edge states across the five seeds). When three to five tasks are at the edge, the cloud is selected because its faster processor more than offsets its longer propagation delay (45 of 45 states). In the intermediate band of one to two tasks, the values of the two offload tiers are close and the choice varies with task size, channel quality and seed: 39 of the 45 such states select the cloud and 6 select the edge. Figure 4.3 shows seed 42, whose two exceptions are small tasks at intermediate channel quality and large tasks at good channel quality. Switching between the two offload tiers according to edge load is the source of the improvement over the fixed baselines. Training converged well before the end of the run: the 50-episode moving average of episode reward stays within 5% of its final level from episode 1,364 for seed 42 (median 1,428, range 1,364–1,602 across the five seeds), as shown in Figure 4.2.

Inspecting the full Q-table (Figure 4.3) shows what the agent did and did not learn, and the tabular representation makes both directly checkable. Training visited 27 of the 36 states in the base state space (75.0%) for every seed, and every action within each of those 27 states was tried at least once, so all 81 reachable state-action pairs are covered. The nine unvisited states are exactly those with six or more tasks at the edge. That condition essentially never arises at the arrival rate studied (edge utilisation approximately 0.41 even if every task is offloaded to the edge), and these states are reported as untrained rather than assigned a policy. Coverage is uneven: for seed 42, every action in the idle-edge states was tried at least 4,714 times, in the one-to-two-task band at least 1,407 times, but in the three-to-five-task band as few as 7 times. Restricting to the 18 states in which every action was tried at least 30 times, $Q(\text{local})$ lies between -1.68 and -1.15 , while $Q(\text{edge})$ and $Q(\text{cloud})$ lie between -0.79 and -0.52 . Local execution is therefore strictly dominated throughout the

trained state space, and no seed selects it in any visited state. The reason is a large asymmetry in magnitude. The mean task requires 550 million CPU cycles, which the 500-million-cycle-per-second device needs 1.1 s to execute, whereas even the poorest channel (about 137 kbit/s) uploads a mean-sized task in about 40 ms. What the agent learns is almost entirely edge load. Task size and channel quality matter only in the one-to-two-task band, where the edge and cloud values are close and the greedy choice differs between seeds. This band is also where the agent loses most ground to the Greedy Heuristic, which resolves the same near-tie task by task from the exact CPU demand and queue count (Section 4.7). A Q-table lets an operator check a claim of this kind directly; an opaque deep-learning policy does not.

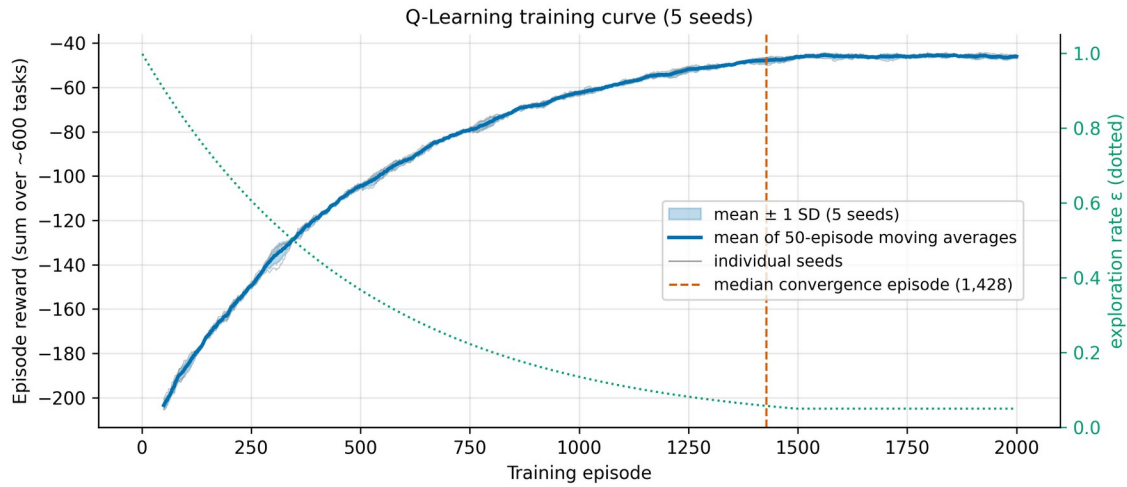


Figure 4.2: Q-learning training curves over 2,000 episodes. The grey lines show the 50-episode moving average of episode reward for each of the five seeds; the blue line is their mean with a ± 1 standard deviation band; the dotted line shows the exploration rate ϵ (right axis). The dashed line marks the median convergence episode (1,428), after which the moving average stays within 5% of its final level (per-seed range 1,364–1,602).

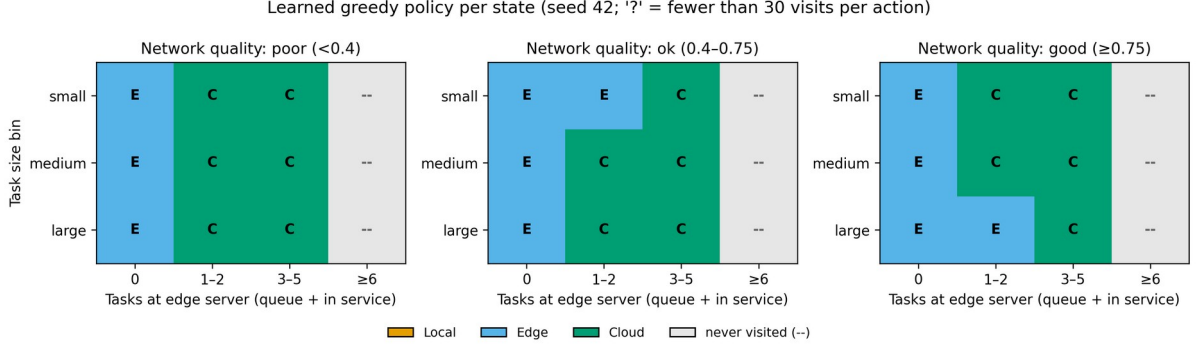


Figure 4.3: Learned greedy policy per state for seed 42 (L = Local, E = Edge, C = Cloud; '?' marks fewer than 30 visits per action; '--' marks states never visited). Columns count the tasks at the edge server, waiting or in service. The policy selects the edge when the edge is idle and the cloud when three or more tasks are present, and the one-to-two-task band contains the only size- and channel-dependent choices. Local execution is never selected. States with six or more tasks at the edge were never visited in training.

4.5 Pareto Trade-Off Analysis

To examine the latency-energy trade-off, the cost weights were swept across seven settings from pure latency to pure energy. A separate agent was trained for each setting (seed 42, 1,500 training episodes, frozen greedy evaluation over 2,000 simulated seconds at quality 0.9), and the resulting operating points were plotted (Figure 4.4). Average latency varies between 0.0821 s and 0.0909 s across the weightings (the highest at $w_{\text{lat}} = 0$, where the agent is indifferent between the two offload tiers), but the frontier is degenerate rather than merely flat. Across all seven weightings the average energy is 1.9439×10^{-4} J, identical to machine precision, because every weighting is evaluated on the same sequence of arriving tasks and each task incurs the same transmission energy whichever offload tier receives it. The cause is structural. In the cost model of Section 3.3, device energy under offloading is the uplink transmission energy, the product of transmission power and transmission time. It depends on task size and channel quality but not on the offload destination: sending a task to the edge and sending it to the cloud cost the device the same energy, because the device performs the same transmission in both cases. Device energy therefore takes only two distinct values per task, one for local execution and one shared by both offload actions. Since local execution is strictly dominated and never selected (Section 4.4), energy is constant over the agent's entire realised action set, and no weighting of a constant can move the operating point along the energy axis. This is an informative negative result about the

problem formulation rather than about the algorithm. It shows that a genuine latency-energy frontier cannot arise from tier selection alone. It requires either device-side continuous control variables such as CPU frequency and transmission power, which change energy without changing the destination, or a model in which the destination itself alters device energy, for example through multi-hop relaying. This directly motivates the continuous-control work proposed in Section 5.3. The sweep uses a single seed; because energy is structurally constant, the degeneracy does not depend on the seed.

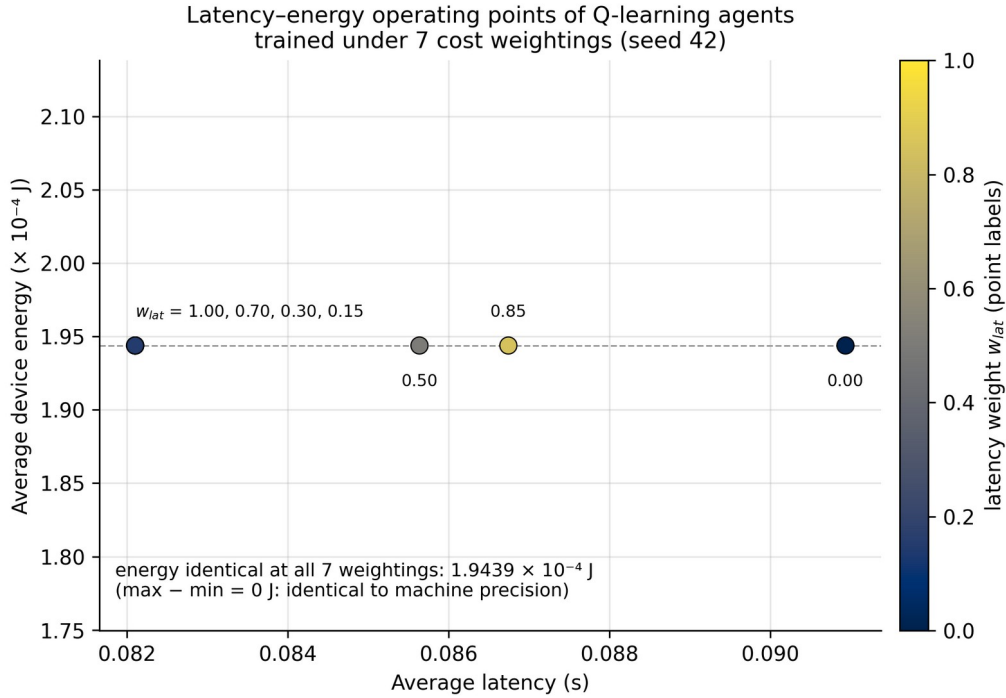


Figure 4.4: Latency-energy operating points of seven Q-learning agents, each trained under a different latency weight w_{lat} (seed 42; point labels give w_{lat} ; the agents for $w_{lat} = 1.00, 0.70, 0.30, 0.15$ learned the same policy and coincide). Average device energy is identical at every weighting, so the points lie on a horizontal line and form a degenerate frontier: device energy does not depend on the offload destination in this model.

4.6 Sensitivity Analysis

To check that the conclusions do not hinge on the single hyperparameter configuration of Table 3.1, the learning rate α , the discount factor γ and the exploration decay factor were each varied one at a time around their defaults, with every other setting unchanged, and the full five-seed protocol was repeated for each variant (Table 4.4). The variants were chosen

before their results were seen, and the reported configuration was not changed afterwards. Mean Q-learning cost ranges from 0.05763 ($\alpha = 0.05$) to 0.05862 (ϵ decay 0.999). Every variant remains significantly cheaper than Always-Cloud, with the whole 95% confidence interval above zero. No variant closes the gap to the Greedy Heuristic; every interval against it lies below zero. The myopic variant $\gamma = 0$, which ignores the successor state entirely, reaches a mean cost of 0.05815 against 0.05856 for the default. Because a decision affects later tasks only through the edge queue, most of the achievable benefit here is immediate, and the problem is close to a contextual bandit. The effect of the discount factor is small compared with the gap to the Greedy Heuristic, which points to the state representation rather than the planning horizon as the limiting factor. A smaller learning rate ($\alpha = 0.05$) lowers the mean cost to 0.05763 and narrows the gap to the heuristic to -4.63% , consistent with the estimation-noise explanation in Section 4.7; its confidence interval overlaps the default's, so this is an observation, not a tuned result, and the reported configuration is unchanged.

Table 4.4: One-at-a-time hyperparameter sensitivity (five seeds per row, 2,000 training episodes, 1,000-second frozen greedy evaluation; mean with 95% confidence interval; negative values mean Q-learning is more expensive; ϵ decay 0.999: 2 of 5 seeds had not met the 5% convergence criterion by episode 2,000).

Setting	Q-learning cost	vs Always-Cloud	vs Greedy Heuristic	Convergence episode (median)
Default (Table 3.1)	0.05856 ± 0.00149	15.60% [13.47%, 17.74%]	-6.32% [-9.08% , -3.57%]	1,428
$\alpha = 0.05$	0.05763 ± 0.00099	16.94% [15.60%, 18.28%]	-4.63% [-6.19% , -3.08%]	1,427
$\alpha = 0.30$	0.05855 ± 0.00202	15.61% [12.72%, 18.51%]	-6.31% [-10.10% , -2.52%]	1,400
$\gamma = 0.0$ (myopic)	0.05815 ± 0.00240	16.20% [12.76%, 19.64%]	-5.56% [-9.51% , -1.62%]	1,474
$\gamma = 0.5$	0.05805 ± 0.00212	16.34% [13.36%, 19.32%]	-5.39% [-9.04% , -1.74%]	1,428
$\gamma = 0.99$	0.05841 ± 0.00312	15.82% [11.40%, 19.64%]	-6.05% [-11.52% , -0.58%]	1,430

		20.24%]	-0.57%]	
ϵ decay 0.995	0.05787 ± 0.00148	16.60% [14.47%,	-5.07% [-7.67%,	602
		18.72%]	-2.47%]	
ϵ decay 0.999	0.05862 ± 0.00200	15.52% [12.65%,	-6.42% [-9.70%,	1,818
		18.39%]	-3.13%]	

4.7 Discussion

What was learned. Q-learning recovers a simple, interpretable load-threshold policy from rewards alone: use the edge while it is idle and the cloud once it is busy, and never compute locally. That policy is 15.60% cheaper than the best fixed tier. The Greedy Heuristic, with full task information and the cost model in closed form, is 20.62% cheaper than Always-Cloud. The agent therefore recovers about 76% of the improvement that the better-informed heuristic achieves, without being given the model.

Where it falls short. The remaining gap has two identifiable sources. The first is information. The heuristic knows each task's CPU demand, which decides whether a busy edge or the cloud finishes the task sooner, while the agent's state omits it; the agent can only learn the average consequence of an action in a state, and the per-task variation is invisible to it. The second is estimation noise in near-tied states. With a constant learning rate the Q-values keep fluctuating, and in the one-to-two-task band the values of edge and cloud are close, so the greedy choice there differs between seeds. The data bear this out. The three seeds whose policy at good channel quality still sends some tasks to a busy edge are 6.87–8.81% more expensive than the heuristic. The two seeds with a pure idle-edge threshold are 3.95–4.07% more expensive. Even the cleaner policy is therefore limited by the missing information, which a richer state (for example, a binned CPU demand) or function approximation would be needed to recover.

A failure in the evaluation protocol. The audit that produced this version found that the first version evaluated the agent while it was still learning. That inflated the headline improvement over Always-Cloud (17.03% before the correction, 15.60% after) and hid most of the seed-to-seed variability. The corrected protocol freezes the Q-table before evaluation, and a unit test now guards against the regression. The lesson generalises: in

reinforcement-learning evaluation, train/evaluation separation must be enforced in code, not by convention.

Robustness to the simulator's timing simplification. Simulating the upload and propagation delays in event time, so that a task joins the edge queue only after its upload, changes the improvement over Always-Cloud from 15.60% to 15.79% and the difference to the heuristic from -6.32% to -4.89% . Neither conclusion changes.

What the results do not show. They do not show that Q-learning outperforms a well-specified analytical policy; it does not. They do not establish a performance ceiling for tabular methods or a crossover point with deep methods, since no deep agent was trained and no optimal-policy bound was computed. They also do not cover other channel qualities, arrival rates or multiple devices. These questions define the thesis phase.

Chapter 5: Conclusion and Future Work

5.1 Summary and Contributions

This project designed, implemented and validated a tabular Q-learning agent for task offloading in a three-tier device-edge-cloud system. The principal contributions are: (1) a reproducible Python/SimPy discrete-event simulator for three-tier offloading, incorporating a Shannon-Hartley wireless channel in which network quality genuinely affects transmission cost, and validated against M/M/1 theory (within 0.39%) and against closed-form end-to-end latencies (within 0.21%); (2) a tabular Q-learning baseline whose policy, and whose training coverage, are both directly inspectable; and (3) an evaluation across five independently trained seeds with the learned policy frozen, showing that Q-learning reduces composite cost by 15.60% (95% CI [13.47%, 17.74%]) relative to the strongest fixed baseline, Always-Cloud ($t(4) = 20.31$, $p = 3.5 \times 10^{-5}$). Measured against Always-Edge, the baseline used to frame the target in the project proposal, the improvement is 19.84% (95% CI [16.68%, 22.99%]), clear of the proposed 10% target. Measured against the Greedy Heuristic, an untrained strategy that is given the cost model and each task's exact parameters, the learned policy is 6.32% more expensive (95% CI 3.57–9.08%; $p = 0.003$). This result is reported as plainly as the others. The value of tabular Q-learning here is not that it exceeds a well-specified analytical policy; a short heuristic with no training and more information does that. Its value is that it recovers most of the heuristic's improvement (about 76%) from interaction alone, without access to the closed-form cost model or the task's CPU demand. The conclusions are robust to one-at-a-time changes of the learning rate, discount factor and exploration schedule (Section 4.6).

A fourth contribution is methodological. The study reports state-space coverage alongside the learned policy rather than assuming every state was learned equally well. That practice, together with a final audit, surfaced three implementation issues, all corrected. The first is an arg-max tie-breaking rule that could let an untried, zero-initialised action outrank a genuinely evaluated one in sparsely visited states. The second is an edge-load observable that omitted the task in service, which merged an idle server and a busy server into one state. Re-measured with the frozen evaluation protocol on seed 42, correcting the observable changes the improvement over Always-Cloud from 9.39% to 14.18% and state-

action coverage from 67 of 78 to 81 of 81. The third is that the evaluation runs kept updating the Q-table. That inflated the headline improvement (17.03% before the correction, 15.60% after) and understated its seed-to-seed variability. The degenerate Pareto frontier reported in Section 4.5 is a separate and more durable structural finding: it is a property of the cost model rather than of the algorithm or of training coverage, and it persists after all three corrections.

5.2 Limitations

The study is bounded by its design choices, and several limitations are worth stating precisely. Tabular Q-learning requires the state to be discretised into bins and does not scale to high-dimensional or continuous states. The state omits the task's CPU demand, so from the agent's perspective the problem is partially observable; this is the main identified reason why the Greedy Heuristic, which sees that demand and is given the cost model, remains cheaper (Section 4.7). Training visited 27 of the 36 base states. The remaining 9 correspond to edge loads that essentially never occur at the arrival rate studied, so the policy is undefined there and is reported as such rather than extrapolated; states with three to five tasks at the edge were visited only rarely. The constant learning rate leaves Q-values fluctuating, so near-tied states can resolve differently from seed to seed. Device energy in the cost model is the same for both offload destinations, so the discrete three-action space admits no latency-energy trade-off for the agent to exploit; a non-degenerate Pareto frontier requires continuous control variables. The model is single-agent and therefore does not capture contention between multiple devices for shared edge resources. Local execution is modelled without device-side resource contention, so tasks assigned to the device execute in unbounded parallel; the reported Always-Local latency of 1.102 s is therefore a lower bound, since a genuine single-CPU device would be unstable at this arrival rate (the offered load on the device alone is approximately 6.6 times its capacity). This strengthens rather than weakens the conclusion that offloading is essential. Uplink and propagation delays are added to latency analytically rather than being simulated in event time; Section 4.7 shows that this does not change the conclusions. The successor state used in the Bellman bootstrap is observed at task completion and reuses the completed task's size, rather than the size of the next arrival, which is unknown at that instant. Because task sizes are drawn independently this introduces a bias in the bootstrap term that is not corrected here. Finally,

all evaluation is conducted at a single fixed channel quality (0.9) with five seeds, the Pareto sweep uses one seed, and all results are obtained in simulation rather than on physical infrastructure.

5.3 Future Work (Thesis Phase)

These limitations define the direction of the subsequent thesis phase, which will characterise empirically where a tabular method's advantages give way to a deep one's:

- State-space scaling study: train a Deep Q-Network (DQN) alongside the tabular Q-learning baseline established here across five state-space complexity levels (36 to 3,240 states, adding task complexity, battery level, task type and finer discretisation in turn) to locate empirically where, if anywhere, a neural approximator's ability to generalise across similar states overtakes an exact table's. Adding the task's CPU demand to the state is the first step, since Section 4.7 identifies it as the main missing information. A preliminary probe exists: adding velocity and network-quality-trend bins to model mobility (Section 3.4.1) expands the space to 324 states, and under the same training budget the tabular method visits only 81 of the 324 states (25.0%) (seed 42, velocity 0.8), consistent with the scaling problem this study is designed to locate. The mobility configuration has not been evaluated for performance.
- Deadline-aware reward shaping: introduce a weighted penalty for missed task deadlines and evaluate, across a swept penalty weight, whether it measurably improves deadline compliance without degrading average latency-energy cost.
- Action masking under congestion: remove the edge-offload action from the agent's choice set whenever the edge queue is already occupied, testing whether preventing exploration of a predictably futile action improves learning efficiency.

The interpretable Q-learning baseline delivered by this project, together with the Greedy Heuristic as a stronger reference point than any fixed strategy, provides the well-characterised basis against which the thesis-phase DQN comparison, deadline-shaping study and masking experiment will be evaluated. Federated multi-agent learning, in which

multiple devices share model updates without exchanging raw task data, is a longer-horizon extension beyond this thesis phase and is not scheduled among the three studies above.

References

- Alfakih, T., Hassan, M. M., Gumaei, A., Savaglio, C., & Fortino, G. (2020). Task Offloading and Resource Allocation for Mobile Edge Computing by Deep Reinforcement Learning Based on SARSA. *IEEE Access*, 8, 54074-54084. <https://doi.org/10.1109/ACCESS.2020.2981434>
- Bi, S., Huang, L., Wang, H., & Zhang, Y.-J. A. (2021). Lyapunov-Guided Deep Reinforcement Learning for Stable Online Computation Offloading in Mobile-Edge Computing Networks. *IEEE Transactions on Wireless Communications*, 20(11), 7519-7537. <https://doi.org/10.1109/TWC.2021.3085319>
- Chen, X., Jiao, L., Li, W., & Fu, X. (2016). Efficient Multi-User Computation Offloading for Mobile-Edge Cloud Computing. *IEEE/ACM Transactions on Networking*, 24(5), 2795-2808. <https://doi.org/10.1109/TNET.2015.2487344>
- Chen, X., & Liu, G. (2022). Federated Deep Reinforcement Learning-Based Task Offloading and Resource Allocation for Smart Cities in a Mobile Edge Network. *Sensors*, 22(13), 4738. <https://doi.org/10.3390/s22134738>
- Dash, S., Mishra, J., & Dash, S. K. (2026). Modified Q-Learning Approach for Optimized Task Offloading and Resource Allocation in Mobile Edge Computing. *SN Computer Science*, 7, Article 99. <https://doi.org/10.1007/s42979-025-04673-z>
- Henderson, P., Islam, R., Bachman, P., Pineau, J., Precup, D., & Meger, D. (2018). Deep Reinforcement Learning That Matters. *Proceedings of the AAAI Conference on Artificial Intelligence*, 32(1). <https://doi.org/10.1609/aaai.v32i1.11694>
- Hwang, R.-H., Lai, Y.-C., & Lin, Y.-D. (2021). Offloading Optimization with Delay Distribution in the 3-tier Federated Cloud, Edge, and Fog Systems. *arXiv:2107.05015*
- Lillicrap, T. P., Hunt, J. J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D., & Wierstra, D. (2016). Continuous Control with Deep Reinforcement Learning. *International Conference on Learning Representations (ICLR 2016)*. *arXiv:1509.02971*

- Mao, Y., You, C., Zhang, J., Huang, K., & Letaief, K. B. (2017). A Survey on Mobile Edge Computing: The Communication Perspective. *IEEE Communications Surveys & Tutorials*, 19(4), 2322-2358. <https://doi.org/10.1109/COMST.2017.2745201>
- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., & Hassabis, D. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540), 529-533. <https://doi.org/10.1038/nature14236>
- Peng, P., Lin, W., Wu, W., Zhang, H., Peng, S., Wu, Q., & Li, K. (2024). A survey on computation offloading in edge systems: From the perspective of deep reinforcement learning approaches. *Computer Science Review*, 53, 100656. <https://doi.org/10.1016/j.cosrev.2024.100656>
- Shi, W., Chen, L., & Zhu, X. (2023). Task Offloading Decision-Making Algorithm for Vehicular Edge Computing: A Deep-Reinforcement-Learning-Based Approach. *Sensors*, 23(17), 7595. <https://doi.org/10.3390/s23177595>
- Shuai, J., & Xie, B. (2024). Q-Learning-Based Task Offloading Strategy for Satellite Edge Computing. *International Journal of Communication Systems*. <https://doi.org/10.1002/dac.5691>
- Sonmez, C., Ozgovde, A., & Ersoy, C. (2018). EdgeCloudSim: An Environment for Performance Evaluation of Edge Computing Systems. *Transactions on Emerging Telecommunications Technologies*, 29(11), e3493. <https://doi.org/10.1002/ett.3493>
- Souza, P. S., Ferreto, T., & Calheiros, R. N. (2023). EdgeSimPy: Python-Based Modeling and Simulation of Edge Computing Resource Management Policies. *Future Generation Computer Systems*, 148, 446-459. <https://doi.org/10.1016/j.future.2023.06.013>

- Sun, J., Zhang, W., Han, M., Shen, M., & Wu, X. (2026). Deep Reinforcement Learning-Based Adaptive Task Offloading for Mobile Edge Computing. *The Journal of Supercomputing*, 82(5), Article 345. <https://doi.org/10.1007/s11227-026-08492-8>
- Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.
- Watkins, C. J. C. H. (1989). *Learning from Delayed Rewards* [PhD thesis]. King's College, University of Cambridge.
- Watkins, C. J. C. H., & Dayan, P. (1992). Q-learning. *Machine Learning*, 8(3-4), 279-292. <https://doi.org/10.1007/BF00992698>
- You, C., Huang, K., Chae, H., & Kim, B.-H. (2017). Energy-Efficient Resource Allocation for Mobile-Edge Computation Offloading. *IEEE Transactions on Wireless Communications*, 16(3), 1397-1411. <https://doi.org/10.1109/TWC.2016.2633522>